Adaptive Prompt Compression via Contextual Bandits (LinUCB)

Abstract

With the proliferation of Large Language Models (LLMs), optimizing API costs while maintaining generation quality has become a critical challenge. This paper presents an adaptive prompt compression framework using the **LinUCB Contextual Bandit** algorithm. By dynamically selecting compression strategies based on linguistic features, the system balances token savings with semantic integrity. Experimental results demonstrate that our approach achieves significant cost reduction while maintaining high response validity, even under extreme resource constraints.

1. Introduction

1.1 Motivation

LLM pricing models are typically token-based and limited by context window sizes. Existing compression methods often rely on static rules, failing to account for the sensitivity of different content types. For instance, whitespace removal might break Python code logic, while stopword filtering is highly effective for casual conversations.

1.2 Objective

Develop an intelligent routing system that dynamically navigates the trade-off between **Cost (Tokens)** and **Quality (Validity)**.

2. Related Work

Recent advancements in prompt engineering have introduced static compression techniques such as *LLMLingua* and *Selective Context*, which utilize information-theoretic metrics to prune low-perplexity tokens. However, these methods often lack real-time adaptability to diverse downstream tasks. Our work bridges this gap by introducing a feedback loop through reinforcement learning, allowing the system to learn category-specific sensitivities autonomously.

3. Methodology

3.1 Theoretical Background: Why Contextual Bandits?

Unlike standard Multi-Armed Bandits (MAB), which assume a stationary reward distribution for each action, **Contextual Bandits (LinUCB)** incorporate an observed feature vector (the context) to inform each decision. In prompt optimization, the "context" consists of linguistic features. LinUCB is chosen for its superior **sample efficiency** and its ability to maintain linear computational complexity, making it ideal for real-time API routing.

3.2 Feature Extraction

Input text x is mapped to a feature vector $f(x)$, including:

- Normalized text length.
- Unique word ratio (lexical diversity).
- Structural features (presence of code keywords like `def`, `{`).
- Average word length.

3.3 Action Space (Arms)

The agent chooses between three strategies (Arms):

- a_0 (Raw): No processing (High quality, high cost).
- a_1 (Basic): Strips extra whitespaces and newlines.
- a_2 (Aggressive): Filters stopwords, retaining only semantic entities (Low cost, higher risk).

3.4 Core Algorithm: LinUCB

We utilize the LinUCB algorithm to handle the **Exploration vs. Exploitation** trade-off. For each arm a, the estimated upper confidence bound $p_{t,a}$ is:

$$p_{t,a} \stackrel{\text{def}}{=} \theta_a^\top x_{t,a} + \alpha \sqrt{x_{t,a}^\top A_a^{-1} x_{t,a}}$$

3.5 Reward Function Design

The reward function is designed to penalize failures heavily:

$$Reward = w_1 \cdot \text{Saving} - w_2 \cdot \text{Latency} - w_3 \cdot \text{FailurePenalty}$$

4. Experimental Setup

- **Model**: Google Gemini 1.5 Flash (via API).
- **Interface**: Interactive Streamlit Dashboard.

- **Constraints**: Evaluated under extreme daily API quotas (20 requests/day) to test **Sample Efficiency**.

5. Results and Discussion

5.1 Sample Efficiency under Quota Constraints

Experimental data shows that despite strict API limits, LinUCB successfully identifies successful signals. The negative reward feedback correctly steers the agent away from risky compression on sensitive data.

5.2 Robustness to Code Sensitivity

The agent learns to assign a_0 or a_1 to snippets identified as "Code," while aggressively applying a_2 to "Chat" categories, optimizing both cost and functionality.

6. Discussion & Limitations

6.1 Risk of Hallucinations

While aggressive compression (a_2) maximizes cost savings, it may occasionally remove vital negations or nuances, leading to model hallucinations. Our reward function mitigates this by applying a heavy penalty to invalid responses.

6.2 Dependency on Stopword Lists

The current implementation of Arm 2 relies on a static English stopword list. Future iterations will explore **Entropy-based Pruning** to support multi-lingual datasets more effectively.

7. Conclusion

This study demonstrates the feasibility of using reinforcement learning for automated prompt management. Future work includes exploring multi-model routing and offline pre-training on large-scale datasets.